



Universidad Autónoma de Baja California

Facultad de Ciencias Químicas e Ingeniería (FCQI)

Monitor de Estado de Servicios UABC

Salazar Leal Javier Isaac - 2208260

Deraz Labrador Emmanuel 2208862

Profesora: Felicitas Perez Ornelas

Índice del Proyecto

-  1. Descripción y Objetivo
-  2. Alcance del Sistema:
-  3. Evidencias de Avance:
-  4. Retos y Soluciones:
-  5. Herramientas:
-  6. Conclusiones
-  7,. Referencias



Descripción y Avance

30%

Avance Actual

(Fase 1 Frontend: 50% | Fase 2 Backend: Iniciando)

🎯 Objetivo Principal

Desarrollar una plataforma centralizada (Dashboard) funcional que permita a la comunidad universitaria verificar en tiempo real el estado de los servidores de la UABC (SIIA, Blackboard, etc.).

📄 Descripción del Proyecto

El sistema se compone de una interfaz frontend construida con tecnologías web que se conecta, a través de una API, con un **Backend en Python** encargado de realizar el monitoreo HTTP real y extraer el estatus de las páginas institucionales.

Alcance del Proyecto



Incluye:

- Diseño Frontend UI/UX (Finalizado).
- Desarrollo de API Backend en Python.
- Peticiones HTTP a servidores **UABC**.
- Registro histórico de fallos.

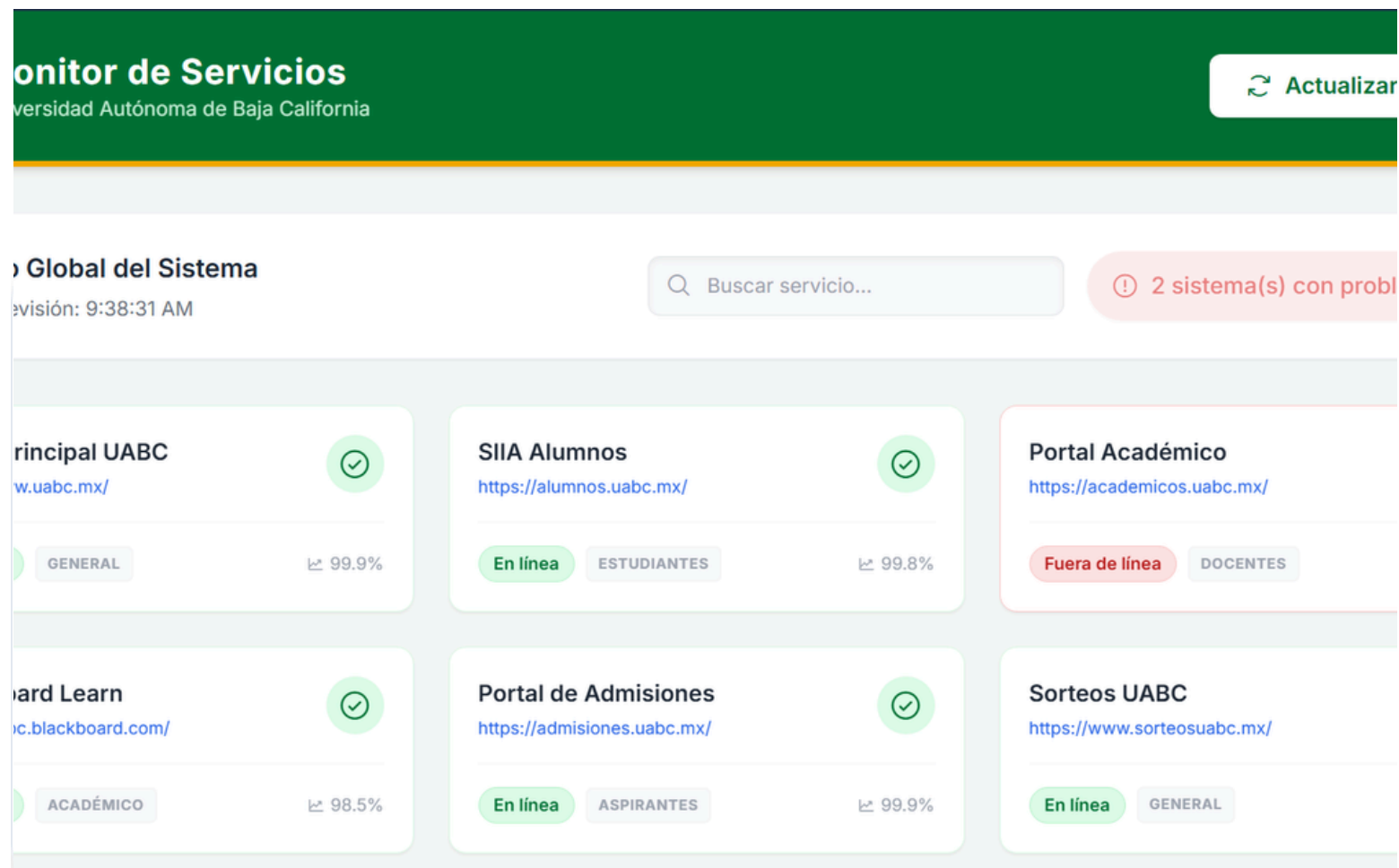


No Incluye:

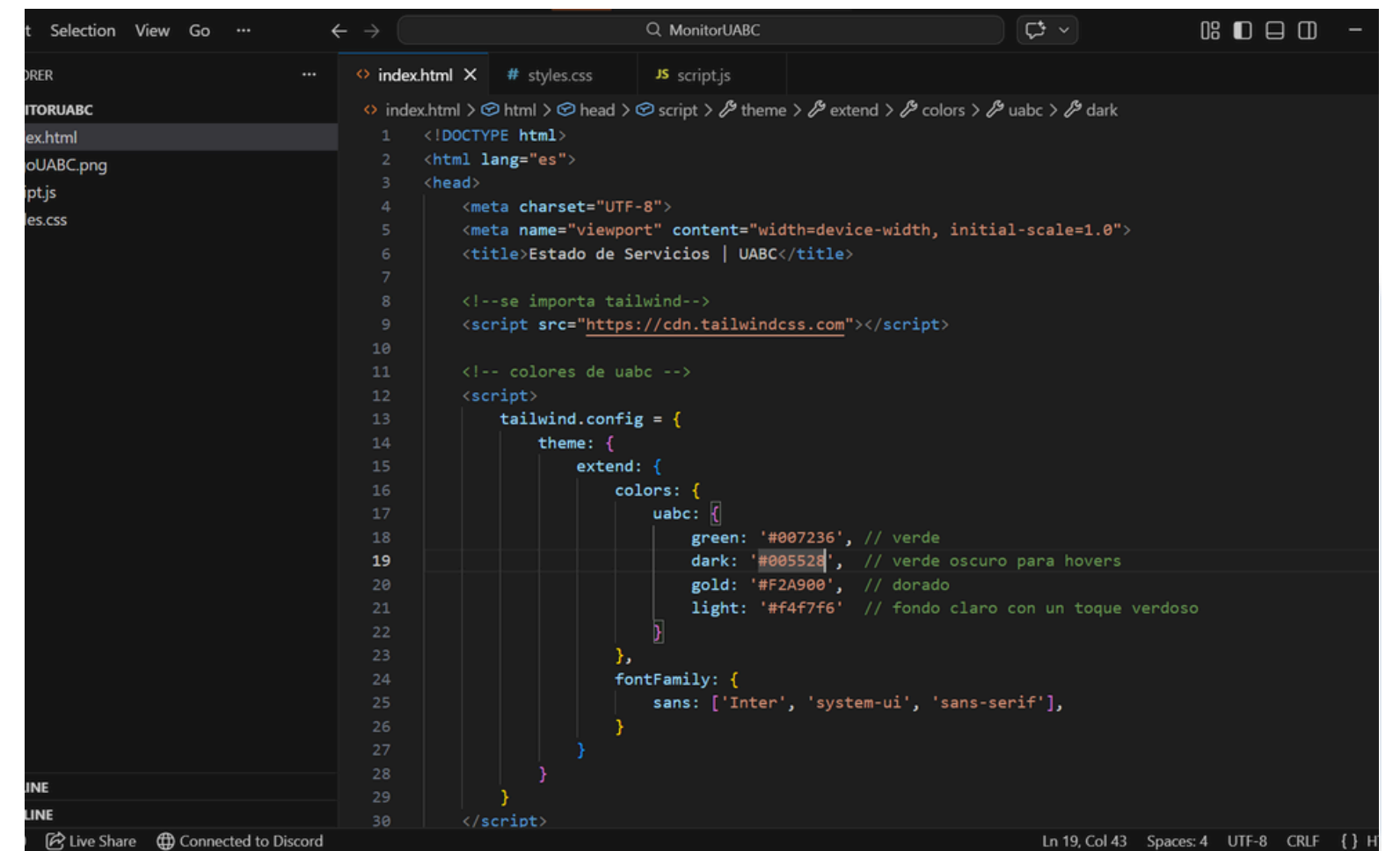
- Modificación o gestión de los servidores propios de la UABC (es estrictamente de "Solo lectura" / Monitoreo).
- Sistema de alertas por SMS al celular.



Evidencias del Avance Actual



1. Interfaz Web (Fase 1)



</> 2. Código Fuente

⚠ Retos y Soluciones

Javier Isaac Salazar Leal

⚠ **Problema 1: Restricción de Conexión**
Hacer "pings" reales directamente desde el navegador web es bloqueado por las políticas de seguridad de la UABC.

✓ **Solución Planteada : Arquitectura Cliente-Servidor**
Creación de una API REST en Python. El backend será el encargado de revisar los sitios web y le enviará la información "limpia" al frontend.

⚠ **Problema 2: Sobrecarga al Servidor UABC**
Si 1000 estudiantes entran al dashboard, se harían 1000 pings por segundo a la UABC, actuando como un ataque DDoS.

✓ **Solución Planteada: Tareas Programadas**
El backend de Python hace la revisión *una vez cada ciertos minutos*, y le muestra la misma caché de datos a todos los usuarios.



Tecnologías y Stack



Python (Backend)

El núcleo del sistema. Utiliza librerías como Requests para hacer llamadas HTTP a la UABC y FastAPI/Flask para exponer los datos vía API.



JavaScript (Lógica UI)

Encargado de consumir la API de Python a través de fetch() en tiempo real y actualizar los colores del dashboard (Verde/Rojo).



Tailwind CSS

Framework empleado para consolidar el diseño de la interfaz, respetando la línea gráfica e institucional de la Universidad.

Conclusiones y Siguietes Pasos

A manera de conclusión, nos gustaría destacar la importancia y eficacia del uso de diversas tecnologías para el desarrollo de software, y cómo estas hacen sinergia para entregar un producto completo. Por otra parte, la prevención de errores mediante el análisis previo a la programación es un aspecto fundamental a considerar, ya que permite ahorrar tiempo en soluciones técnicas, como se observó en este caso con la restricción del ping y el reconocimiento de ataques DDoS por parte de UABC para nuestra aplicación de status.

El siguiente paso consiste en implementar la lógica de la página en el backend, dado que actualmente solo contamos con la interfaz y una simulación de pings para obtener su estado. Asimismo, se deberá vincular esta lógica con el frontend e implementar un registro (log) de fallos basado en la respuesta recibida por el backend

Referencias

-  Python, R. (2021, julio 28). Python and REST APIs: Interacting with web services. Recuperado el 24 de marzo de 2026, de Realpython.com website: <https://realpython.com/api-integration-in-python/>
-  Tailwind Labs. (2026). Styling with utility classes. Tailwind CSS. <https://tailwindcss.com/docs/styling-with-utility-classes>
-  Quickstart — requests 2.33.0.dev1 documentation. (s/f). Recuperado el 24 de marzo de 2026, de Readthedocs.io website: <https://requests.readthedocs.io/en/latest/user/quickstart/>
-  Requests: HTTP for humans™ — requests 2.33.0.dev1 documentation. (s/f). Recuperado el 24 de marzo de 2026, de Readthedocs.io website: <https://requests.readthedocs.io/en/latest/>